Intern Project: AI-Powered Service Request Bot

Oct 22, 2026 · @Moustafa

Overview

Interns will build a Telegram bot that lets residents report maintenance issues by text or voice note, then turns each report into a categorized, stored service request (SR) shown on an insights dashboard. The project runs 10 weeks and covers the full AI lifecycle: conversational intake, LLM extraction, ML classification, data storage, memory design, deployment and analytics.

Objectives

- Deliver a working end-to-end prototype, demoable to stakeholders in 4 weeks
- Give each intern hands-on experience with LLMs, speech-to-text, classical ML, databases, dashboards and cloud deployment.
- Compare an LLM-based approach with a trained ML classifier, with measured results.
- Use only public or synthetic data; no internal Emaar data is required.

Scope and functional requirements

The system has five functional parts: intake, extraction, classification, storage and the dashboard.

User flow

1. The user opens the Telegram bot and describes the issue in text or a voice note (Arabic or English).
2. Voice notes are transcribed to text (speech-to-text).
3. The LLM extracts structured fields from the conversation: unit information (compound, building, unit number), contact details and the issue description.
4. If fields are missing or unclear, the LLM asks follow-up questions until the request is complete.
5. The bot shows a summary and asks the user to confirm.

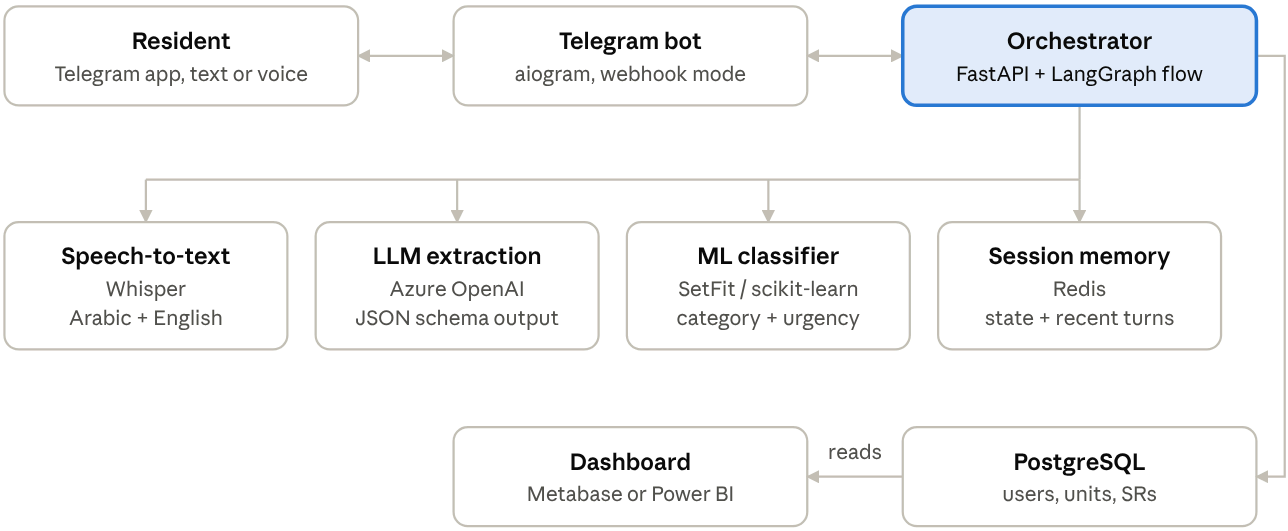
6. The ML classifier assigns a category (plumbing, electricity, AC, carpentry, pest control, general) and an urgency level.
7. The SR is saved to the database with a ticket number, and the user receives confirmation.
8. The SR appears on the dashboard.

Functional requirements

Component	Requirement
Telegram intake	Accept text and voice notes; support Arabic and English; handle multi-turn conversations
LLM extraction	Return validated structured output (JSON schema) for unit info, contact and issue; ask for missing fields
ML classification	Trained classifier for category and urgency; return a confidence score; low-confidence cases flagged for review
Database	Store users, units, SRs, conversation history and model predictions with timestamps
Memory	Remember returning users and their units; keep conversation state per chat (see Memory design)
Dashboard	SR volume over time, category and urgency breakdown, top units and buildings, open vs closed, average handling time
Status	Users can check their SR status by ticket number in the bot

Architecture and tech stack

One orchestrator connects the bot, AI services and storage



system architecture · 8 components

The orchestrator runs each conversation as a LangGraph flow: transcribe if needed, extract fields, ask for missing ones, confirm, classify, then save the SR to PostgreSQL. The dashboard reads directly from the database.

Layer	Technology
Bot	Python, aiogram (or python-telegram-bot)
Speech-to-text	Whisper (open-source, self-hosted) or Azure AI Speech
LLM	Azure OpenAI with structured outputs; LangGraph for the conversation flow
Classifier	TF-IDF + logistic regression baseline, then SetFit or multilingual BERT; MLflow for experiment tracking
API	FastAPI
Storage	PostgreSQL (persistent data), Redis (session memory)
Dashboard	Metabase or Power BI

Layer	Technology
Ops	Docker, GitHub Actions, Azure Container Apps, Langfuse for LLM tracing

Memory design

Memory is split into three layers, each with its own storage and lifetime, so the bot stays accurate without sending unbounded history to the LLM.

Layer	What it holds	Storage	Lifetime
Session state	The current SR being filled: collected fields, missing fields, conversation step	Redis, keyed by Telegram chat ID	Cleared on submit, or after 30 minutes idle
Conversation history	Recent messages of the active session, passed to the LLM	Redis, last N turns only; older turns summarized	Same as session
User profile	Name, phone, registered units, past SR IDs	PostgreSQL	Persistent

Rules the design must follow

- Returning users are recognized by Telegram user ID; the bot offers their saved unit ("Is this for Unit B-204?") instead of asking again.
- The LLM never receives other users' data; the prompt includes only the current user's profile and session.
- Unit information is confirmed by the user before it is saved to their profile; the LLM's extraction is never written without confirmation.
- History is trimmed or summarized to stay within a fixed token budget per call.
- A user can type /reset to clear the session, and /forget to delete their saved profile.
- Personal data (phone, name) is stored only in the database, not in logs or traces.

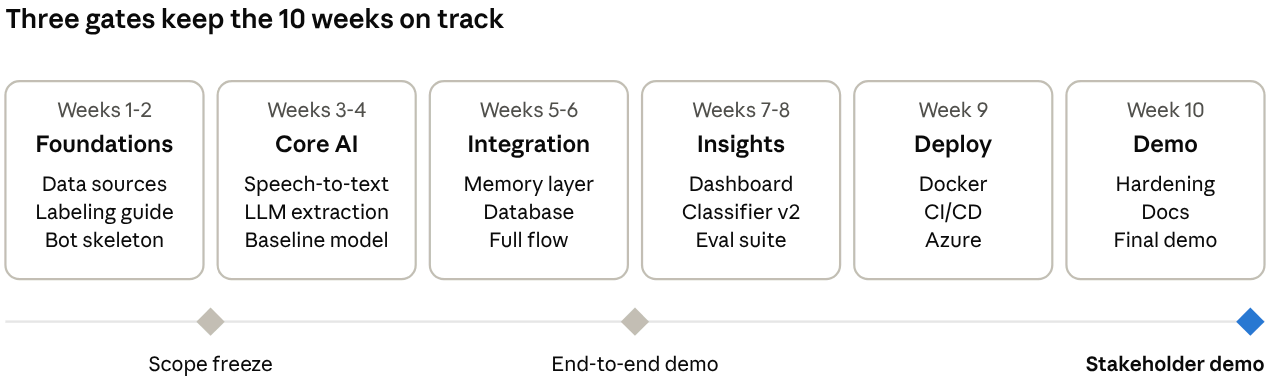
Data strategy

No internal Emaar data is used; interns build the training and test data from public and synthetic sources, and building it is part of the learning.

Need	Source
Training data for the classifier	NYC 311 Service Requests (NYC Open Data), filtered to housing complaints and mapped to the project categories
Arabic and Egyptian-style requests	LLM-generated synthetic requests modeled on the 311 data, plus a translated subset
Test set	200–300 requests written and labeled by the interns themselves, kept out of training
Voice notes	Interns record a small set of Arabic and English voice notes to test transcription
Units and users	A mock unit registry (compounds, buildings, unit numbers) generated by the team

Every source and its license is recorded in a data card. If the prototype moves forward later, the same pipeline can be pointed at real data under internal access controls.

Timeline, team and deliverables



10-week plan · 6 phases, 3 gates

The week-6 gate is the key checkpoint: one request must flow from Telegram to the dashboard, even if every part is basic.

Team split (4 interns)

Role	Owns
Data and ML	Dataset, data card, classifier training, MLflow, ML vs LLM comparison
LLM and speech	Whisper setup, extraction prompts and schema, follow-up questions
Backend and memory	FastAPI, LangGraph flow, Redis session memory, PostgreSQL schema
Dashboard and DevOps	Dashboard, Docker, CI/CD, Azure deployment, Langfuse tracing

All interns review each other's pull requests, so everyone understands the full system.

Deliverables

- [ ] Git repository with README and setup instructions
- [ ] Data card listing every source and license
- [ ] Model report: classifier results and the ML vs LLM comparison
- [ ] Deployed Telegram bot and live dashboard
- [ ] Final demo and a short written report

Success criteria, out of scope and risks

Success criteria

- Category classification F1 of 0.85 or higher on the held-out test set.
- Extraction captures all required unit fields in at least 90% of test conversations.
- Arabic voice-note transcription is usable for extraction in at least 85% of test recordings.
- Bot reply time under 5 seconds at p95 (excluding voice transcription).
- A written comparison of the ML classifier vs an LLM zero-shot classifier on accuracy, latency and cost per request.
- Live deployment, with a 10-minute demo to stakeholders.

Out of scope

- Integration with real Emaar systems (CRM, work-order or facility tools).
- Assigning technicians or scheduling visits.
- Authentication against real resident records.

Risks

Risk	Mitigation
Synthetic data doesn't reflect real requests	Hand-written test set; report the gap between synthetic and real-style performance
Egyptian Arabic voice transcription is weak	Test Whisper sizes early (week 2); fall back to asking the user to confirm the transcript
LLM extracts wrong unit info	Mandatory user confirmation before saving; schema validation
Scope creep	Freeze scope after week 2; extras go to a stretch list